

DNA Analyzer Tool Working - Detailed Technical Guide

Date: 2026-03-22

Project: DNA Analyzer (frontend + Flask backend)

1. What this document covers

This document explains, tool-by-tool:

- How each result is produced (input -> processing -> output fields)
 - Which values are computed from real logic
 - Which values are hardcoded or static knowledge tables
 - Which outputs can be estimated/fallback rather than strict thermodynamic/biological calculations
 - Where network AI is used vs local generation
-

2. System architecture summary

Frontend flow

- UI tabs collect user input, validate/sanitize, and call API endpoints.
- Some tools also run local analytics in browser JavaScript.
- Certain modules include fallback behavior when API fails.

Backend flow

- Flask API in backend/app.py exposes:
 - /api/mutations
 - /api/parse-hgvs
 - /api/align
 - /api/crispr
 - /api/primers
 - /api/ai-explain
 - /api/explain (legacy)
 - /api/health
 - /api/info
- Core algorithms are pure Python modules in backend/algorithms.

Security/ops controls

- CORS allowlist is hardcoded for specific origins.
 - Rate limit is in-memory: 30 requests/minute/IP.
 - AI key is read from environment variable GROQ_API_KEY.
-

3. Shared data validation and preprocessing

DNA cleaning

Across endpoints/components, sequences are typically:

- converted to uppercase
- whitespace/newline removed
- validated against allowed alphabet

Backend allows A/T/G/C/N in validation.

Frontend helper validateSequence in apiUtils removes everything except A/T/G/C before sending in many flows.

Length gates (hard constraints)

- General validation max length: 100,000 bp
- Alignment max length: 10,000 bp
- Primer design min: 50 bp, max: 50,000 bp

These are strict hardcoded limits.

4. Tool-by-tool deep dive

4.1 Overview / DNA Sequence Analyzer (local browser analysis)

Primary logic: `src/utils/dnaUtils.js`

What is computed

- `cleanSeq()`: removes non-ATGC characters
- `gcContent()`, `atContent()`, `nucleotideCounts()`
- `meltingTemp()`:
 - Wallace rule for short sequences (<14)
 - Empirical formula for longer sequences
- `reverseComplement()`
- `findORFs()` on forward + reverse strands (3 frames each)
- `translate()` via hardcoded codon table
- `calculateCodonUsage()`
- `translateAllFrames()`
- `findRestrictionSites()` for predefined enzymes
- `calculateMolecularWeight()` using fixed per-base mass constants
- `summary()` aggregates all metrics

How result is formed

`summary()` returns object:

- `length`
- `gc`, `at`
- `tm`
- `molecularWeight`
- `nORFs`, `allORFs`, `longestORF`
- `revcomp`
- `nucleotides`
- `codonUsage`
- `sixFrameTranslation`
- `restrictionSites`

Hardcoded/static parts

- Codon table is static in file
- Restriction enzyme list is static (EcoRI, BamHI, etc.)
- Molecular-weight constants are static approximations

Any fake output?

- No random/mock values in core summary.
 - Results are deterministic from sequence and formulas.
 - Biological interpretation text in UI can be generated as fallback prose (local rule-based text), not model-based inference.
-

4.2 Mutation Finder (Cancer/Research modes)

Primary logic: src/components/MutationFinder.jsx + backend/algorithms/mutation_finder.py + backend/algorithms/hgvs_parser.py

Core backend mutation engine

find_mutations(seq1, seq2) in Python:

- Pads shorter sequence with '-' to compare equal length
- Scans position-wise:
 - SNP when both bases present and differ
 - insertion/deletion when one side is '-'
- For SNP, computes codon context and classifies as Silent/Missense/Nonsense where possible
- Returns:
 - summary counts
 - mutation list with position/type/reference/alternate/class

Cancer intelligence in frontend

MutationFinder component includes large embedded knowledge structures:

- gene panel metadata (TP53, BRCA1, KRAS, EGFR)
- domain maps and descriptions
- COSMIC hotspot subset table
- ClinVar representative entries
- hotspot residue lists
- AA property table
- BLOSUM62 matrix for SIFT-like estimate
- Embedded short PDB text for TP53 and links for structures

These improve annotation/visualization but are static data inside code.

HGVS parsing path

Backend /api/parse-hgvs:

- Parses protein HGVS (e.g., R175H, Arg175His, E746_A750del)
- Parses simple cDNA substitutions (c.524G>A)
- Maps cDNA to amino-acid change using stored canonical CDS references

How result is formed (high level)

- Raw sequence or HGVS input is normalized
- Variant object is constructed (position, ref/alt AA, mutation_type)
- Domain/context mapping layers add interpretation and evidence labels
- UI merges mutation metrics + annotation cards + structural view data

Hardcoded/static parts

- All cancer knowledge tables in component are static snapshots
- Canonical references are embedded subsets in frontend and backend modules
- Some sample datasets are pre-curated for demos

Any fake output?

- Not fabricated per request; however:
 - Cancer evidence is static embedded subset, not live COSMIC/ClinVar query
 - Some scores are "SIFT-like"/"PolyPhen-like" approximations, not official model outputs
 - 3D view is synthetic/representative visualization, not full atomistic simulation

4.3 Sequence Alignment

Primary logic: backend/algorithms/alignment.py + src/components/SequenceAlignment.jsx

Algorithms used

- Needleman-Wunsch (global)
 - default scoring: match +1, mismatch -1, gap -2
- Smith-Waterman (local)
 - default scoring: match +2, mismatch -1, gap -1

How result is formed

Backend returns:

- alignment1, alignment2
- score
- stats from calculate_alignment_stats:
 - matches, mismatches, gaps, length, similarity_percentage
- algorithm display name

Frontend then adds interpretation layer:

- detects conserved blocks (≥ 10 matches contiguous)
- detects variable blocks (≥ 5 mismatch/gap contiguous)
- assigns confidence bucket using similarity + gap frequency

Hardcoded/static parts

- Scoring parameters are hardcoded defaults
- Frontend sample scenarios are predefined strings
- Interpretation thresholds are fixed heuristic boundaries

Any fake output?

- Alignment itself is real dynamic-programming output.
- "Biological interpretation" is heuristic narrative generated from rule thresholds.

4.4 CRISPR PAM Finder

Primary logic:

- backend/algorithms/crispr.py
- src/components/CRISPRFinder.jsx

Backend implementation

find_pam_sites(sequence, pam_pattern='NGG', guide_length=20):

- Converts PAM with IUPAC N -> regex [ATGC]
- Scans forward strand for PAM hits
- Builds guide from upstream region (for default SpCas9 logic)
- Scans reverse complement similarly
- Estimates efficiency by simple rule:
 - High if GC in 40-60 and no poly-T stretch
 - Medium / Low otherwise
- Returns site list + forward/reverse/total counts

Frontend implementation

CRISPRFinder supports multiple Cas systems and custom PAM in browser:

- SpCas9, SaCas9, Cas12a, custom
- IUPAC mapping to regex
- Computes guide/context and simple efficiency labels
- Can call AI explanation endpoint

Hardcoded/static parts

- Cas enzyme definitions are fixed constants
- Sample sequence is fixed
- Efficiency model is heuristic (not deep predictive model)

Any fake output?

- PAM search is real pattern matching.
 - Efficiency category is estimated heuristic, not experimentally validated predictive model.
-

4.5 PCR Primer Designer

Primary logic:

- backend/algorithms/primer_design.py
- src/components/PrimerDesigner.jsx
- src/utils/primerEvalEngine.js

This is the most complex tool and includes both strict computation and fallback layers.

Backend design engine

Key methods:

- calculate_tm_nearest_neighbor() (SantaLucia-style NN with assumptions)
- check_hairpin() (approximation)
- check_primer_dimers() (complementarity-based estimate)
- check_gc_clamp()
- evaluate_primer_quality() scoring with penalties
- _rank_primer_pair() weighted pair ranking
- _build_pair_explanation()
- design_primers() orchestrates candidate generation and ranking

Frontend evaluation engine

PrimerEvalEngine includes:

- NN thermodynamic tables
- Tm, GC, hairpin, self-dimer, cross-dimer estimators
- Complexity validation and auto-rejection gates
- Risk tier derivation from success probability
- Structure scoring and interpretation

Fallback cascade behavior (important)

When strict thermodynamic computation is unstable/unavailable:

- Engine returns guarded fallback values to prevent UI crashes
- Cross-dimer may switch to complementarity-based estimate range
- Melting curve analysis may be simulated from available Tm/GC/product length
- UI flags estimated values in output strings

How result is formed

Final object combines:

- recommended forward/reverse primers
- per-primer metrics (Tm, GC, dG metrics, quality)
- pair metrics (Tm diff, cross-dimer risk, product size)
- specificity analysis
- melting curve summary
- structure/risk/success probability classifications
- optimization suggestions/triggers

Hardcoded/static parts

- Thermodynamic constants/tables are static constants in code
- Scoring weights and thresholds are fixed
- Curated sample templates are hardcoded
- Some "PASS/FAIL" sample scenarios are intentionally constructed demos

Any fake output?

- No random fabricated values by default.
- But some outputs are explicitly estimated/simulated when full thermo model is out-of-bounds:
 - estimated dimer delta-G ranges
 - simulated melting-curve summaries
 - fallback confidence buckets

These are not lab measurements and should be treated as computational guidance.

4.6 Primer Evaluator (separate component)

Primary logic: `src/components/PrimerEvaluator.jsx` + `src/utils/primerEvalEngine.js`

What it does

- Accepts explicit primer pair input
- Runs `evaluatePrimerPair()` to classify:
 - RECOMMENDED / CONDITIONALLY ACCEPTED / REJECTED (and related labels)
- Outputs:
 - weighted score
 - risk tier and success probability
 - specificity, melting behavior, structure score
 - safety triggers and optimization suggestions
 - alternative candidates via `generateAlternatives()`

Hardcoded/static parts

- Risk color maps and display thresholds are fixed constants
- Sample primer presets are static

Any fake output?

- Same caveat as Designer: robust deterministic heuristics + fallback estimates for unstable thermo cases.
-

5. AI explanation tools

Endpoints:

- `/api/ai-explain` (new/recommended)
- `/api/explain` (legacy)

How AI result is formed

- Backend builds prompt context from tool outputs
- Sends to Groq chat completion API
- Model used: llama-3.3-70b-versatile
- Returns assistant text as explanation

Hardcoded/static parts

- Prompt templates are hardcoded per tool type

- Model name, temperature, max_tokens are hardcoded
- System prompt role is fixed bioinformatics-expert framing

Any fake output?

- AI text is generated by external LLM and can be plausible but not guaranteed factual.
 - If API unavailable, some components produce local fallback explanation text.
-

6. Explicit list: what is real vs estimated vs static

Fully computed from input (deterministic)

- Sequence cleaning/counting/GC/AT/reverse complement
- Needleman-Wunsch and Smith-Waterman matrices + traceback
- PAM regex scanning positions
- Basic mutation diffing (SNP/ins/del)
- ORF scanning from codons

Heuristic/computationally estimated

- Primer hairpin/dimer risk scoring
- CRISPR efficiency labels
- Mutation pathogenicity-like proxy scores in frontend
- Alignment biological confidence narration
- Primer success probability and risk tiers

Static/hardcoded knowledge

- Codon tables, enzyme lists, thermodynamic constants
- Gene/domain metadata and evidence subsets
- COSMIC/ClinVar representative datasets in UI
- CORS origins, rate limits, endpoint config defaults
- Sample sequences and demo templates

Fallback/simulated behavior

- Local AI fallback explanation in Overview
 - Primer thermodynamic fallback objects when model fails bounds
 - Estimated cross-dimer ranges and melting-curve text
 - Offline/local report generation when AI endpoint fails
-

7. Potentially misleading points (important transparency)

1. "PDF export" in some components is actually HTML generated in browser and printed/saved as PDF by the browser print dialog.
 2. Cancer evidence tables are embedded snapshots, not live database pulls.
 3. "SIFT-like" and "PolyPhen-like" terms represent approximations, not official tool outputs.
 4. Thermodynamic fallback values are safety-oriented placeholders/estimates, not direct experimental thermochemistry.
 5. AI-generated explanations are language-model output, not ground-truth scientific validation.
-

8. Hardcoded constants inventory (selected)

- API base URL fallback:
 - <https://dna-analyzer-1-ipxr.onrender.com>
- CORS allowed origins list in Flask

- Rate limiting:
 - window: 60 seconds
 - max: 30 requests per window per IP
 - Alignment scoring defaults:
 - global: +1 / -1 / -2
 - local: +2 / -1 / -1
 - Primer defaults:
 - target_tm=60
 - primer_length=20
 - product_size_range=[200, 500]
 - AI defaults:
 - model: llama-3.3-70b-versatile
 - max_tokens: 1500
 - temperature: 0.7
-

9. Reliability statement

- For educational and preliminary research use, this pipeline is strong and transparent.
 - For clinical decisions or publication-grade claims, use validated external pipelines and database-backed annotation workflows.
 - Treat all heuristic/estimated outputs as decision support, not definitive biological truth.
-

10. Source map (major files used)

Backend

- backend/app.py
- backend/algorithms/alignment.py
- backend/algorithms/mutation_finder.py
- backend/algorithms/crispr.py
- backend/algorithms/primer_design.py
- backend/algorithms/hgvs_parser.py

Frontend

- src/utils/dnaUtils.js
- src/utils/primerEvalEngine.js
- src/utils/apiUtils.js
- src/utils/config.js
- src/components/OverviewTab.jsx
- src/components/MutationFinder.jsx
- src/components/SequenceAlignment.jsx
- src/components/CRISPRFinder.jsx
- src/components/PrimerDesigner.jsx
- src/components/PrimerEvaluator.jsx

End of guide.

Appendix A — Method-level backing and reliability (per tool)

This appendix provides function-level details, input/output shapes, algorithmic complexity, edge-cases, and an explicit reliability statement for each major tool so you can trace provenance and trustworthiness.

A.1 Overview / DNA Sequence Analyzer

- Primary file: ``src/utls/dnaUtils.js``
- Key functions & behavior:
 - ``cleanSeq(sequence: string) -> string``: regex sanitizer; $O(n)$. Removes non-ATGC characters; deterministic.
 - ``gcContent(sequence: string) -> number``: simple count; $O(n)$. Returns percentage.
 - ``meltingTemp(sequence: string) -> number``: Wallace for $\text{length} < 14$; empirical approximation for longer sequences; $O(n)$. Assumptions on salt and oligo concentration are embedded constants.
 - ``findORFs(sequence: string) -> ORF[]``: scans 3 frames per strand; detects start/stop codons; $O(n)$.
 - ``translate(codingSequence: string) -> string``: codon lookup using static codon table.

Reliability: low-level numeric outputs (counts, revcomp) are fully reliable. T_m and molecular-weight values are approximations; expect experimental deviation. ORF/translation is reliable for canonical code only.

A.2 Mutation Finder

- Primary files: ``backend/algorithms/mutation_finder.py``, ``backend/algorithms/hgvs_parser.py``, ``src/components/MutationFinder.jsx``
- Key functions:
 - ``find_mutations(ref_seq, alt_seq) -> List[Variant]``: pads shorter sequence and scans position-wise; $O(m)$ where $m = \text{max length}$. Detects SNPs and simple indels. Not a gapped-alignment approach.
 - ``classify_variant(variant, cds_map) -> {type, aa_change}``: maps variant to CDS/codon context using embedded canonical references; errors if mapping missing.
 - ``parse_hgvs(hgvs_str)``: lightweight parser for common HGVS forms; not fully HGVS-compliant.

Reliability: sequence-level diffs are reliable for direct comparisons. Complex indels and structural variants are not robustly handled. Functional classifications depend on accurate CDS mappings; front-end pathogenicity proxies are heuristic only.

A.3 Sequence Alignment

- Primary file: ``backend/algorithms/alignment.py``
- Key functions:
 - ``align_global(s1, s2, scoring)``: Needleman–Wunsch DP matrix; $O(n*m)$ time/memory. Produces optimal alignment under given scoring.
 - ``align_local(s1, s2, scoring)``: Smith–Waterman; $O(n*m)$.
 - ``calculate_alignment_stats(a1, a2)``: linear scan to compute matches/gaps/mismatches.

Reliability: algorithms are exact for the chosen scoring scheme. Biological interpretation requires appropriate scoring and domain knowledge. For very long sequences, performance constraints may affect usability.

A.4 CRISPR PAM Finder

- Primary file: ``backend/algorithms/crispr.py`` and ``src/components/CRISPRFinder.jsx``
- Key functions:
 - ``pam_to_regex(pam_pattern) -> regex``: maps IUPAC to regex.
 - ``find_pam_sites(sequence, pam_regex, guide_length)``: regex scan and guide extraction; $O(n)$.
 - ``estimate_efficiency(guide_seq)``: heuristic rules (GC window, poly-T, simple positional preferences).

Reliability: PAM detection is deterministic. Efficiency calls are heuristic and should be validated experimentally or with specialized prediction tools for design-critical choices. Off-target analysis is out of scope unless integrated separately.

A.5 PCR Primer Designer

- Primary file: ``backend/algorithms/primer_design.py``, supporting ``src/utls/primerEvalEngine.js``

- Key functions:

- ``design_primers(target, constraints) -> PrimerPair[]``: enumerates candidates via sliding windows, filters and ranks; complexity depends on target length and primer-length windows.

- ``calculate_tm_nearest_neighbor(seq)``: uses nearest-neighbor "G/"H corrections; valid within thermodynamic bounds typically for 15–35 nt.

- ``check_hairpin(seq)`` / ``check_primer_dimers(p1,p2)``: short k-mer estimates or heuristic penalties.

Reliability: Tm estimates are reasonably accurate within NN model bounds when salt/conc assumptions match experiments. Dimer/hairpin "G predictions are approximations u flags, not definitive lab outcomes. Specificity checks are limited unless a genomic search is enabled.

A.6 Primer Evaluator

- Primary file: ``src/utls/primerEvalEngine.js``

- Behavior: aggregates per-primer and pair metrics into a weighted score; fallback to Wallace rule for short sequences. Outputs qualitative advice and alternative suggestions.

Reliability: consistent internal scoring; final recommendations are opinionated. Use in-silico PCR or BLAST for high-confidence specificity checks prior to synthesis.

A.7 AI Explanation Endpoint

- Primary file: ``backend/app.py`` (caller) and prompt templates in code

- Behavior: ``build_prompt(tool_output, flags)`` injects estimated/flagged fields into templates and calls external LLM via configured API. Returned text is passed through to UI.

Reliability: LLM output is non-deterministic and may hallucinate. The system tries to mitigate this by including provenance (which values were estimated) in prompts; nevertheless, cross-check AI text against computed metrics and static tables.

If you want, I will integrate these appendix sections into the PDF and regenerate the file now. I can also run a small test suite (unit checks described above) to produce a validation report alongside the PDF — which would you prefer next (regenerate PDF only, or regenerate + run validations)?